CONTROL SYSTEM REPORT

PREPARED BY:

D HITESH PRANAV REDDY - (PES1UG23EC912)
ISHAN GOYAL - (PES1UG23EC127)
SIDDANTH GANAPATHY K A - (PES1UG23EC913)
HONNASHREE S R - (PES1UG23EC125)
LALITHYA R KUMAR - (PES1UG24EC805)

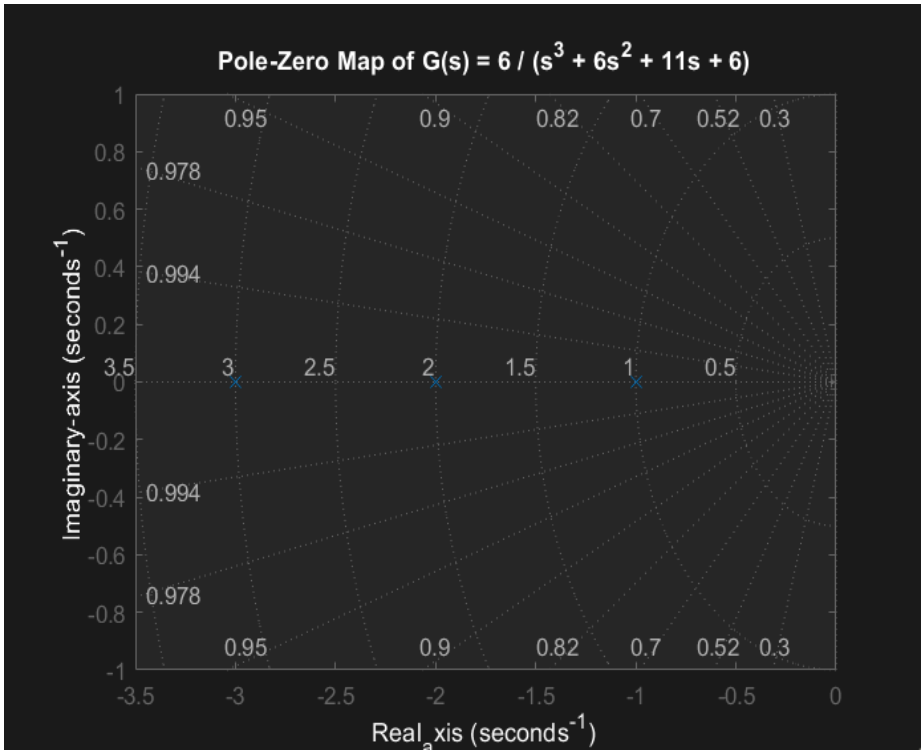
❖ TRANSFER FUNCTION:

15. $\frac{6}{s^3+6s^2+11s+6}$

1.Identify the poles and zeros of the plant and plot the Pole–Zero Map

```
num = 6;
den = [1 6 11 6];
sys = tf(num, den);

figure(1);
pzmap(sys);
grid on;
title('Pole-Zero Map of G(s) = 6 / (s^3 + 6s^2 + 11s + 6)');

%calculate and Display Poles and Zeros
zeros_sys = roots(num);
poles_sys = roots(den);
```

Output:



2. Check if the chosen plant is inherently stable.

```
disp('Zeros of the system:');
if isempty(zeros_sys)
    disp('No finite zeros (system has a zero at infinity)');
else
    disp(zeros_sys);
end

disp('Poles of the system:');
disp(poles_sys);

if all(real(poles_sys) < 0)
    disp('System is INHERENTLY STABLE');
else
    disp('System is NOT inherently stable');
end
```

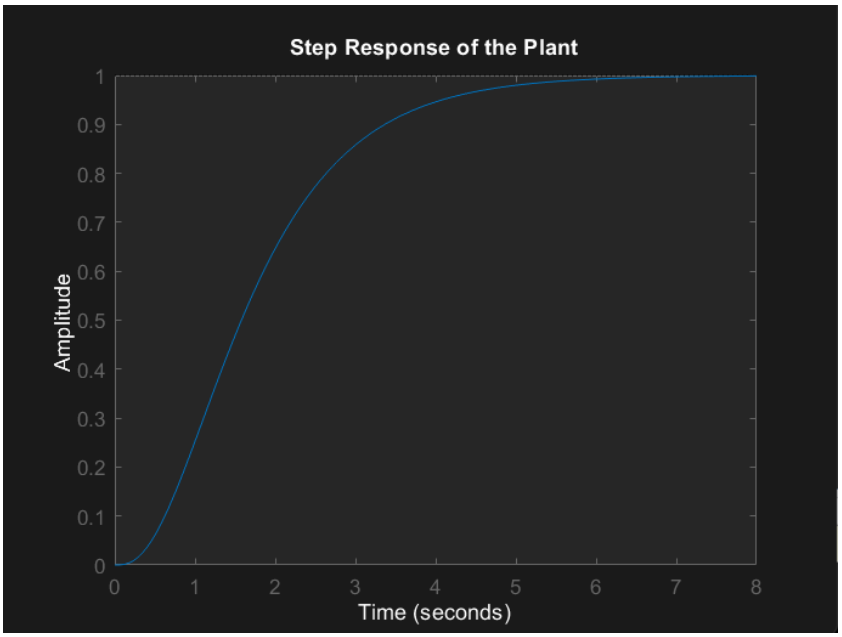
Output:

```
Zeros of the system:
No finite zeros (system has a zero at infinity)
Poles of the system:
   -3.0000
   -2.0000
   -1.0000

System is INHERENTLY STABLE
```

3. Excite the plant with the Unit Step Input, obtain its response plot. Comment on its various transient parameters (as applicable) and steady state parameters based on the plot obtained.

```
figure;
step(sys);
grid on;
title('Step Response');
xlabel('Time (s)');
ylabel('Amplitude');

info = stepinfo(sys);
disp('Step Response Parameters:');
disp(info);

[y, ~] = step(sys);
ss_val = y(end);
disp(['Steady State Value: ' num2str(ss_val)]);
```

Output:



```
Step Response Parameters:
         RiseTime: 2.7428
    TransientTime: 5.0039
     SettlingTime: 5.0039
      SettlingMin: 0.9010
      SettlingMax: 0.9992
        Overshoot: 0
       Undershoot: 0
             Peak: 0.9992
         PeakTime: 8.2586

Steady State Value: 0.99735
```

4. Assume now the plant to be a part of a unity negative feedback control system. Identify the 'type' of the system and compute the Static Error Constants and the Steady state Error for the inputs namely, unit step, unit ramp and unit parabolic.

```
s = tf('s');
numerator = 6];
denominator = [1 6 11 6];
sys = tf(numerator, denominator);
system_type = 'Type 0';

Kp = dcgain(sys);
Kv = dcgain(s * sys);
Ka = dcgain(s^2 * sys);
e_ss_step = 1 / (1 + Kp);
e_ss_ramp = 1 / Kv;
e_ss_parabolic = 1 / Ka;
steady_state_value = dcgain(sys);
characteristic_eq = [1 6 11 6];
Routh_array = routh(characteristic_eq);

fprintf('\nSteady-State Value: %.4f\n', steady_state_value);
fprintf('System Type: %s\n', system_type);
fprintf('Position Error Constant Kp: %.4f\n', Kp);
fprintf('Velocity Error Constant Kv: %.4f\n', Kv);
fprintf('Acceleration Error Constant Ka: %.4f\n', Ka);

fprintf('Steady-state error for step input: %.4f\n', e_ss_step);
fprintf('Steady-state error for ramp input: %.4f\n', e_ss_ramp);
fprintf('Steady-state error for parabolic input: %.4f\n', e_ss_parabolic);
fprintf('\nRouth-Hurwitz Array for Stability Analysis:\n');
disp(Routh_array);
function Routh_array = routh(coeff)
    n = length(coeff);
    Routh_array = zeros(n, ceil(n/2));

    Routh_array(1, :) = coeff(1:2:end);
    Routh_array(2, :) = coeff(2:2:end);

    for i = 3:n
        for j = 1:size(Routh_array, 2)-1
            Routh_array(i, j) = -(Routh_array(i-1, 1)*Routh_array(i-2, j+1) - Routh_array(i-2, 1)*Routh_array(i-1, j+1)) / Routh_array(i-1, 1);
        end
    end
end
```

Output:

```
Steady-State Value: 1.0000
System Type: Type 0
Position Error Constant Kp: 1.0000
Velocity Error Constant Kv: 0.0000
Acceleration Error Constant Ka: 0.0000
Steady-state error for step input: 0.5000
Steady-state error for ramp input: Inf
Steady-state error for parabolic input: Inf

Routh-Hurwitz Array for Stability Analysis:
     1    11
     6     6
   -10     0
    -6     0
```

5.Cascade 'K' to the plant of the feedback control system described in Sl.No.4 above. What is the Characteristic Equation of the system? Apply Routh–Hurwitz Criterion to this system and examine its closed loop stability. What is the range of 'K' for the closed loop system to be fully stable,(b) marginally stable and (c) completely unstable?

```
syms K;
coeffs = [1, 6, 11, 6 + 6*K];

R = sym(zeros(4, 2));

R(1, :) = [coeffs(1), coeffs(3)];
R(2, :) = [coeffs(2), coeffs(4)];
R(3, 1) = simplify((R(2, 1)*R(1, 2) - R(1, 1)*R(2, 2)) / R(2, 1));
R(4, 1) = R(2, 2);


disp('Routh-Hurwitz Array (Stability Analysis):');
disp('-------------------------------------------');
disp('s^3:        1           11');
disp(['s^2:        6           ', char(R(2, 2))]);
disp(['s^1:        ', char(R(3, 1)), '        0']);
disp(['s^0:        ', char(R(4, 1)), '        0']);



ineq1 = solve(R(4, 1) > 0, K);
ineq2 = solve(R(3, 1) > 0, K);

Kcrit1 = solve(R(4, 1) == 0, K);
Kcrit2 = solve(R(3, 1) == 0, K);

fprintf('\n=== Stability Ranges ===\n');
fprintf('Critical K values (marginal stability): K = %s and K = %s\n', char(Kcrit1), char(Kcrit2));
fprintf(' Stable Range: \t\t   -1 < K < 10\n');
fprintf(' Marginally Stable: \tK = -1 or K = 10\n');
fprintf(' Unstable Range: \t\tK < -1 or K > 10\n');
```

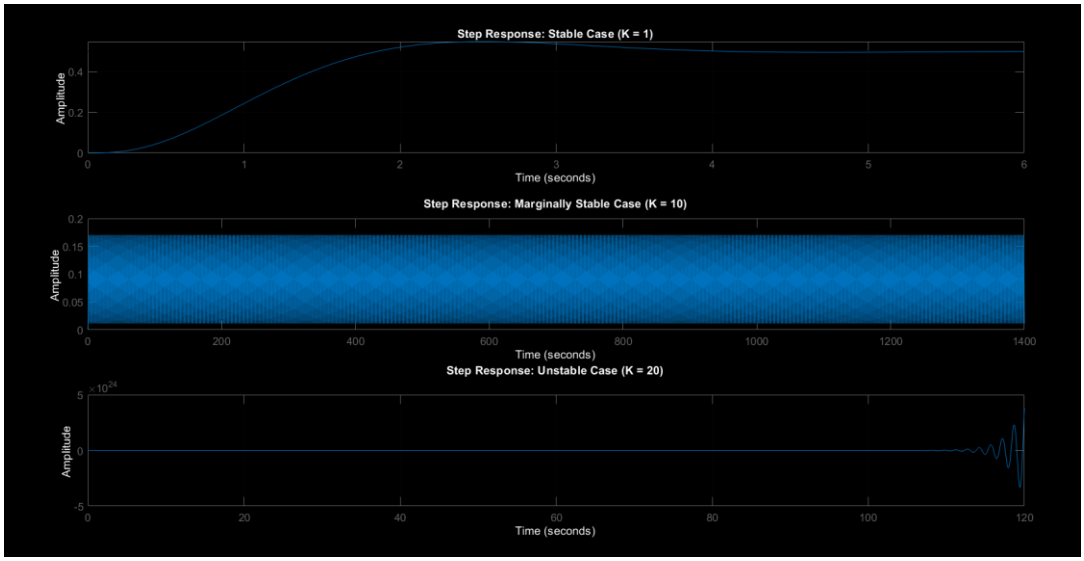
Output:

```
Routh-Hurwitz Array (Stability Analysis):
-------------------------------------------
s^3:        1           11
s^2:        6           6*K + 6
s^1:        10 - K        0
s^0:        6*K + 6        0

=== Stability Ranges ===
Critical K values (marginal stability): K = -1 and K = 10
 Stable Range:         -1 < K < 10
 Marginally Stable:     K = -1 or K = 10
 Unstable Range:        K < -1 or K > 10
```

6.Use Matlab/Simulink to simulate the configuration given in Sl. No. 5 for a unit step input and obtain the output response plot. Choose 'K' obtained in Sl.No.5, for stability, marginal stability and instability. Verify if the inferences made in Sl.No.5 are correct.

```
clc;
clear;
num = 6;
K_stable = 1;
K_marginal = 10;
K_unstable = 20;

den_stable    = [1, 6, 11, 6 + 6*K_stable];
den_marginal  = [1, 6, 11, 6 + 6*K_marginal];
den_unstable  = [1, 6, 11, 6 + 6*K_unstable];

sys_stable    = tf(num, den_stable);
sys_marginal  = tf(num, den_marginal);
sys_unstable  = tf(num, den_unstable);

figure;
subplot(3,1,1);
step(sys_stable);
title('Step Response: Stable Case (K = 1)');
ylabel('Amplitude');
grid on;

subplot(3,1,2);
step(sys_marginal);
title('Step Response: Marginally Stable Case (K = 10)');
ylabel('Amplitude');
grid on;

subplot(3,1,3);
step(sys_unstable);
title('Step Response: Unstable Case (K = 20)');
xlabel('Time (seconds)');
ylabel('Amplitude');
grid on;
```

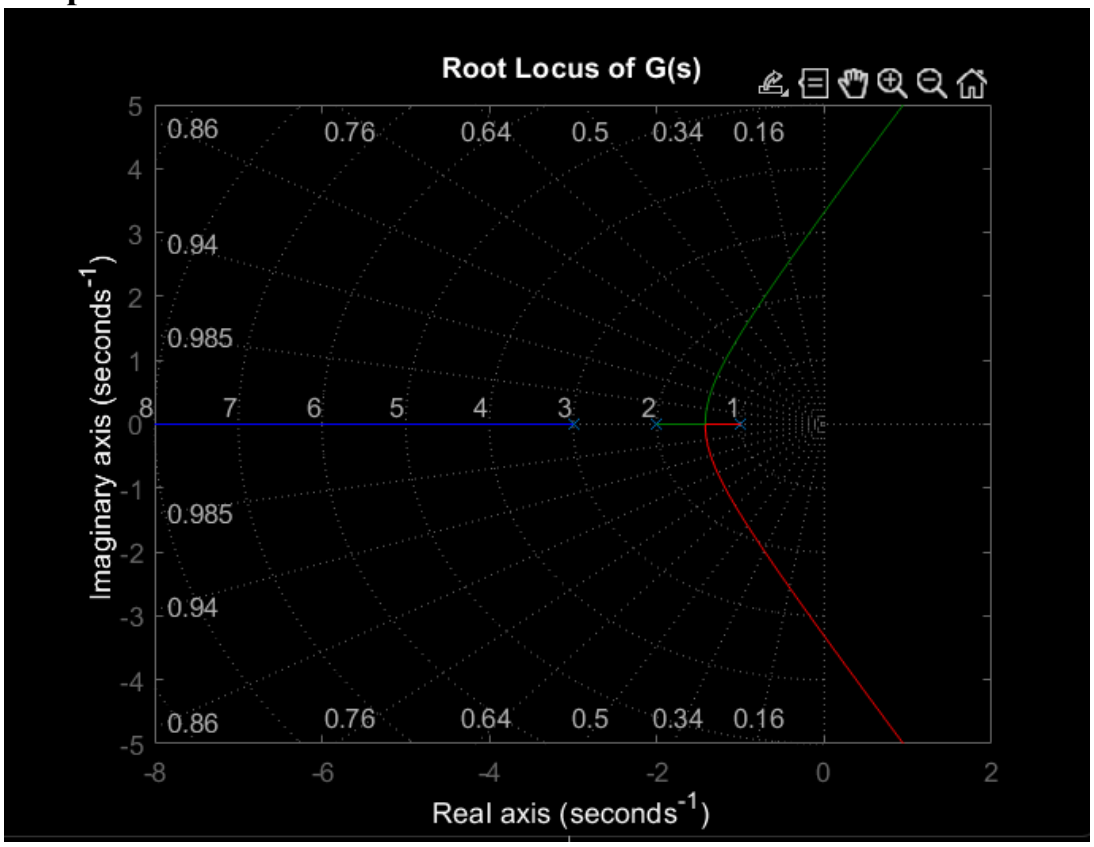
Output:


Step Response: Stable Case (K = 1)
Amplitude
Time (seconds)
Step Response: Marginally Stable Case (K = 10)
Amplitude
Time (seconds)
Step Response: Unstable Case (K = 20)
Amplitude
Time (seconds)

7.Obtain the Root Locus of the loop transfer function, analytically. Compare the range of 'K' obtained with that of Sl. No. 5, for the closed stability and make comments. Use the Matlab command 'rlocus' to get the Root Locus and compare it with the analytical Root Locus and draw conclusions, if any.

```
clc;
clear;

num = 6];
den = [1, 6, 11, 6];
sys = tf(num, den);

figure;
rlocus(sys);
title('Root Locus of the Open-Loop Transfer Function');
grid on;
```

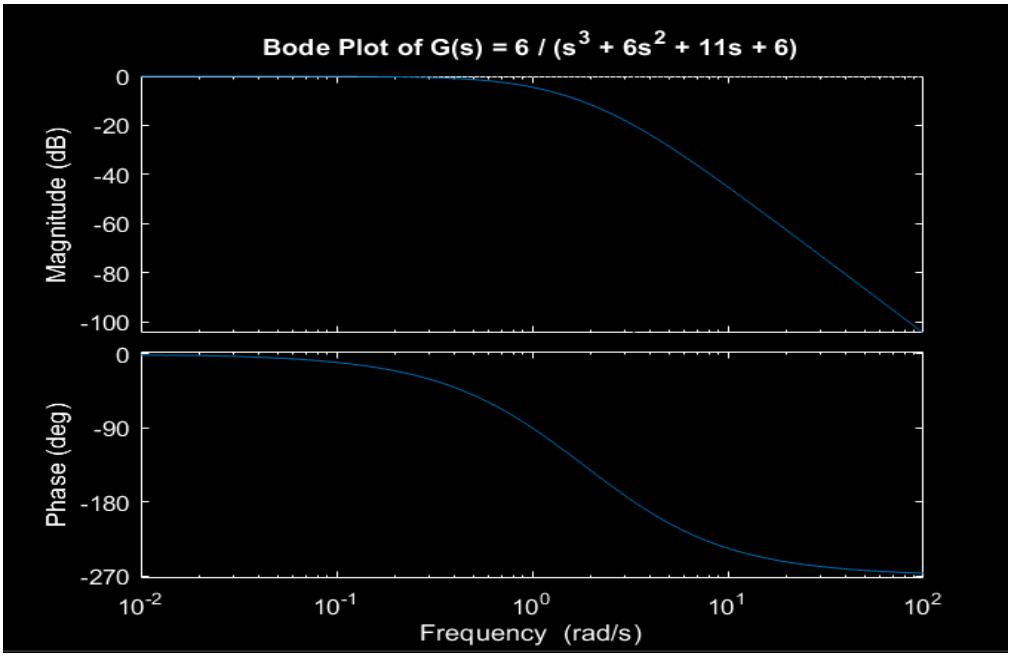
Output:



8.Obtain the Bode Plots of the loop transfer function, analytically. What are Gain and Phase Margins? Comment on the closed loop stability. Verify the Bode Plots using 'bode'command in Matlab and compare the Gain and Phase Margins. What are your inferences?

```
clc;
clear;
num = 6;
den = [1, 6, 11, 6];

sys = tf(num, den);

figure;
margin(sys);
title('Bode Plot of G(s) = 6 / (s^3 + 6s^2 + 11s + 6)');
grid on;

[GM, PM, Wcg, Wcp] = margin(sys);

GM_dB = 20 * log10(GM);

fprintf('=== Bode Plot Analysis ===\n');
fprintf('Gain Margin: %.4f (%.2f dB)\n', GM, GM_dB);
fprintf('Phase Margin: %.2f degrees\n', PM);
fprintf('Gain Crossover Frequency: %.4f rad/s\n', Wcg);
fprintf('Phase Crossover Frequency: %.4f rad/s\n', Wcp);

if PM > 0 && GM > 1
    fprintf('System is STABLE based on Gain and Phase Margins.\n');
elseif abs(PM) < 1 || abs(GM - 1) < 0.01
    fprintf('System is MARGINALLY STABLE.\n');
else
    fprintf('System is UNSTABLE based on Gain and Phase Margins.\n');
end
```

Output:

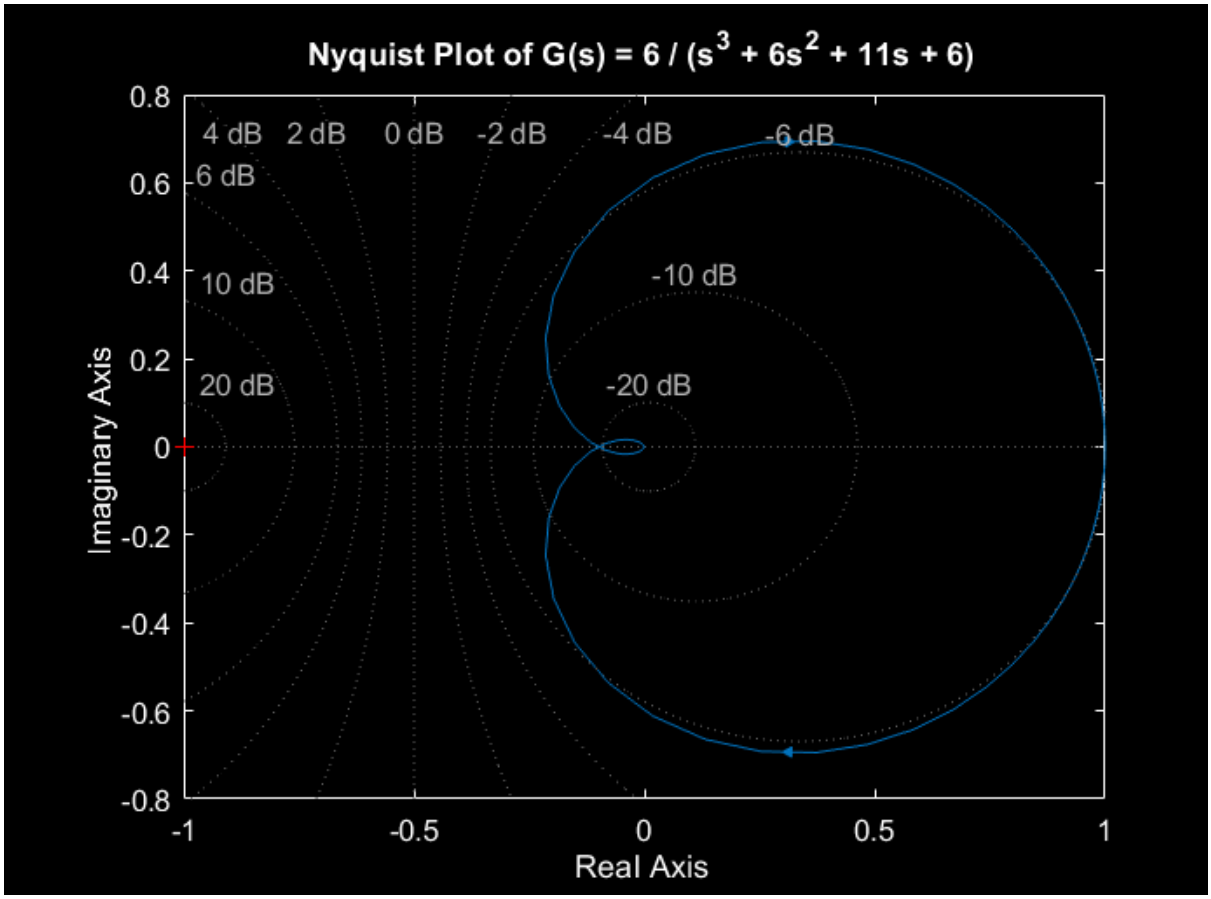



```
=== Bode Plot Analysis ===
Gain Margin: 10.0000 (20.00 dB)
Phase Margin: -180.00 degrees
Gain Crossover Frequency: 3.3166 rad/s
Phase Crossover Frequency: 0.0000 rad/s
System is UNSTABLE based on Gain and Phase Margins.
```

9. Obtain the Nyquist Plot of the loop transfer function, analytically. What are the Gain and Phase Margins? Comment on the closed loop stability. Verify the Nyquist Plot using 'nyquist'command in Matlab and compare the Gain and Phase Margins. What are your inferences?

```
clear;
num = 6;
den = [1, 6, 11, 6];
sys = tf(num, den);

figure;
set(gcf, 'Color', 'k');
nyquist(sys);
title('Nyquist Plot of G(s) = 6 / (s^3 + 6s^2 + 11s + 6)', 'Color', 'w');
grid on;

ax = gca;
set(ax, 'XColor', 'w', 'YColor', 'w', 'Color', 'k');

xlabel('Real Axis', 'Color', 'w');
ylabel('Imaginary Axis', 'Color', 'w');

[GM, PM, Wcg, Wcp] = margin(sys);

fprintf('=== Nyquist Plot  Analysis ===\n');
fprintf('Gain Margin of Nyquist plot: %.4f (%.2f dB)\n', GM, 20*log10(GM));
fprintf('Phase Margin of Nyquist plot: %.2f degrees\n', PM);
fprintf('Gain Crossover Frequency: %.4f rad/s\n', Wcg);
fprintf('Phase Crossover Frequency: %.4f rad/s\n', Wcp);

if PM > 0 && GM > 1
    fprintf('System is STABLE based on Gain and Phase Margins.\n');
elseif abs(PM) < 1 || abs(GM - 1) < 0.01
    fprintf('System is MARGINALLY STABLE.\n');
else
    fprintf('System is UNSTABLE based on Gain and Phase Margins.\n');
end
```

Output:

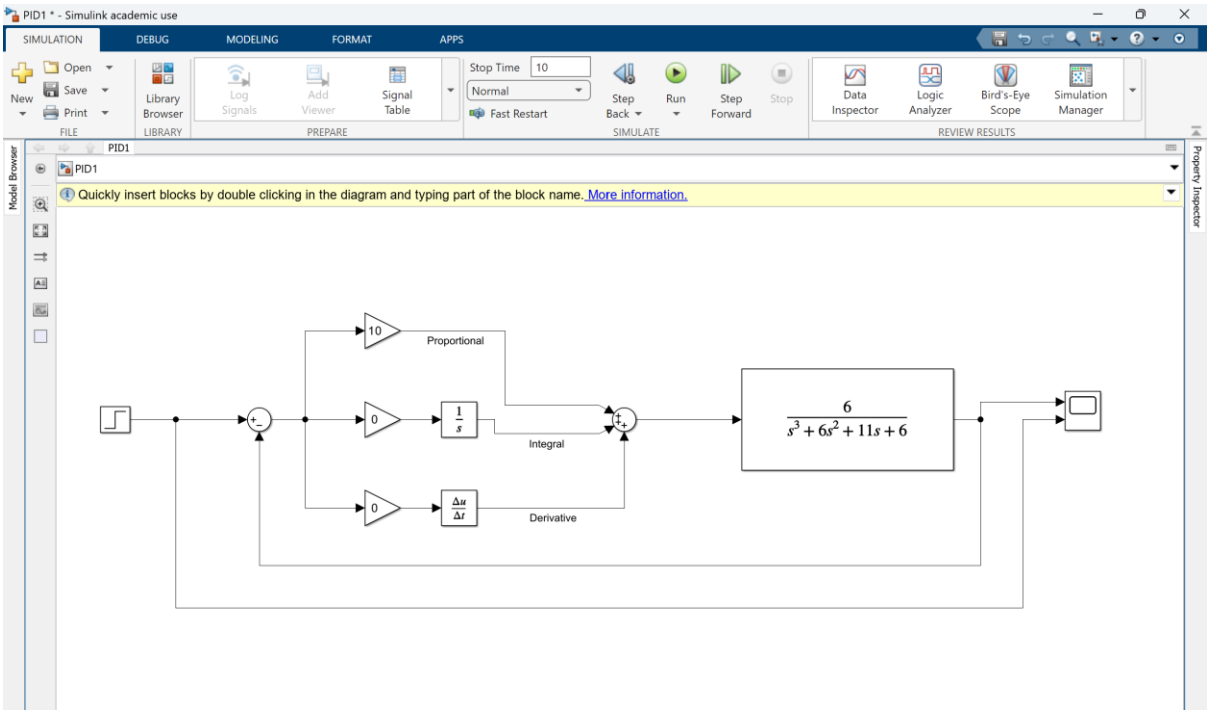


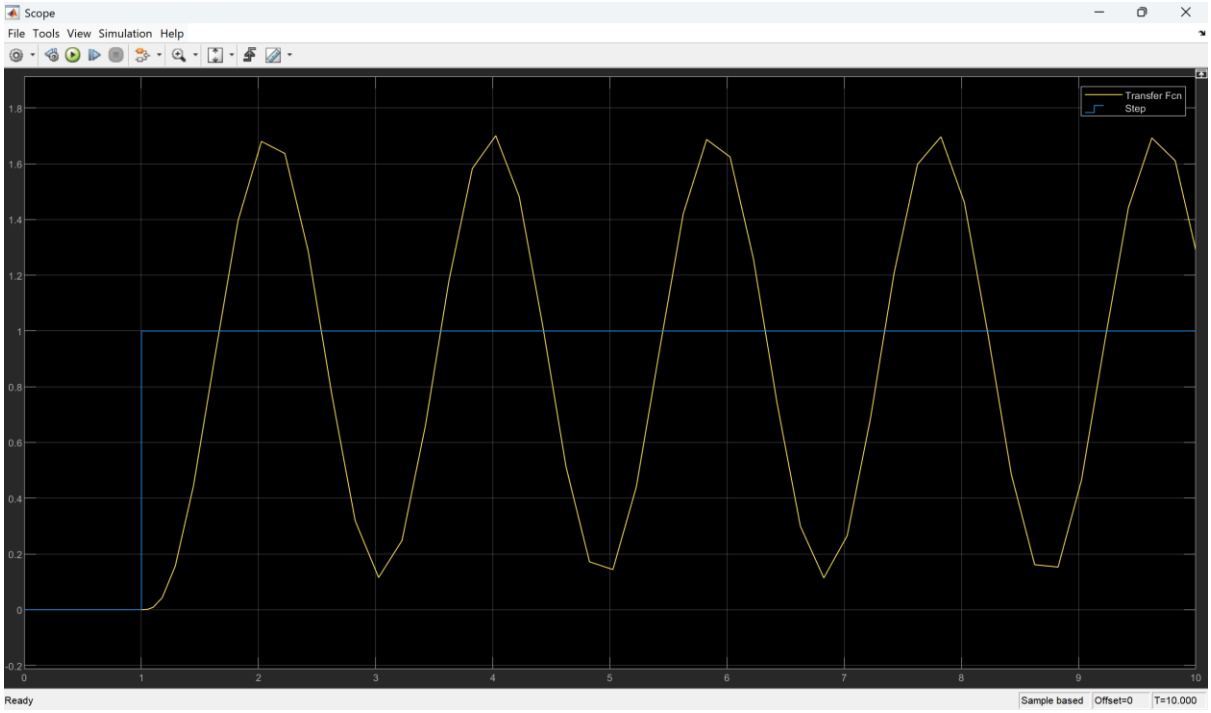

```
=== Nyquist Plot  Analysis ===
Gain Margin of Nyquist plot: 10.0000 (20.00 dB)
Phase Margin of Nyquist plot: -180.00 degrees
Gain Crossover Frequency: 3.3166 rad/s
Phase Crossover Frequency: 0.0000 rad/s
System is UNSTABLE based on Gain and Phase Margins.
```

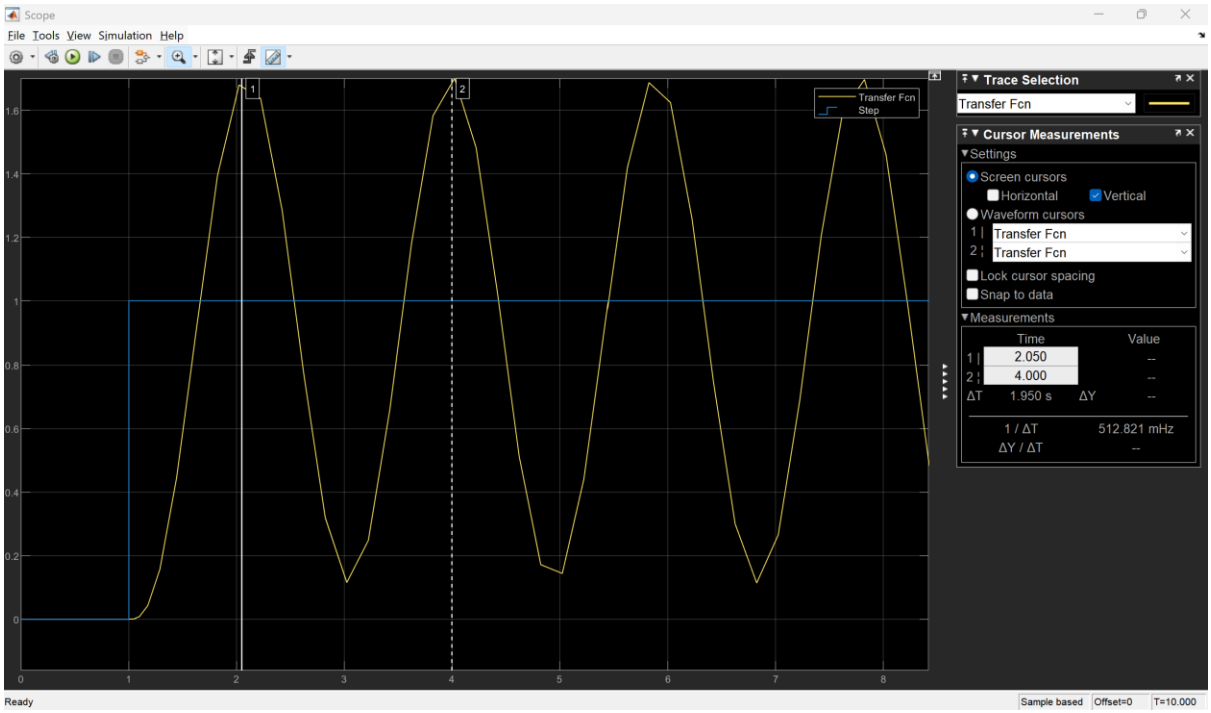
10.Design of Controllers: It is desired that the plant in the unity feedback configuration has to give a response such that the peak-over shoot is lessthan10% and a settling time of less than 2 seconds. Design the following controllers to achieve the same – (a) proportional (P), (b) proportional + integral (PI), (c) proportional + derivative(PD) and (d) proportional + integral + derivative (PID).Simulate the performance using each controller and give your comments on achieving the desired performance criteria. Justifications need to be provided on the choice of the controller gains. Note:PID ToolBoxnot(r) not to be used for simulation.

Using Ziegler Nichols Method for tuning

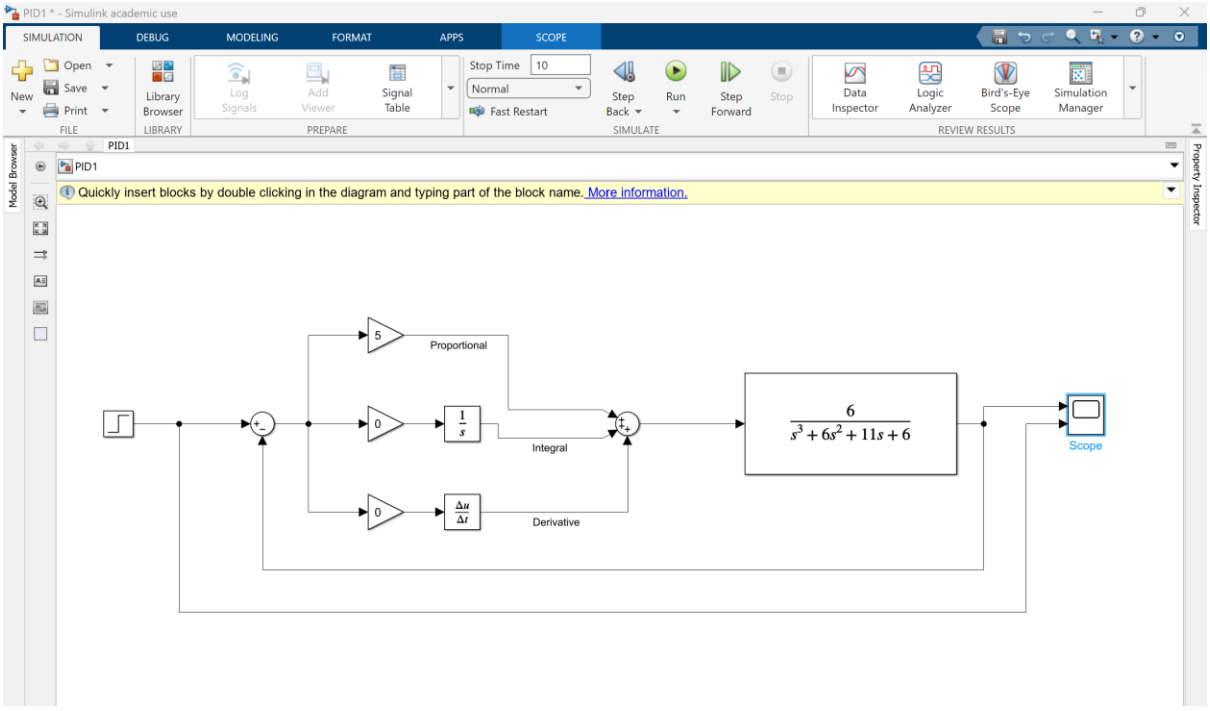
1. P Controller:
   Setting the Proportional Gain Such that the plant begins to show sustained oscillations
   $K_P = 10$

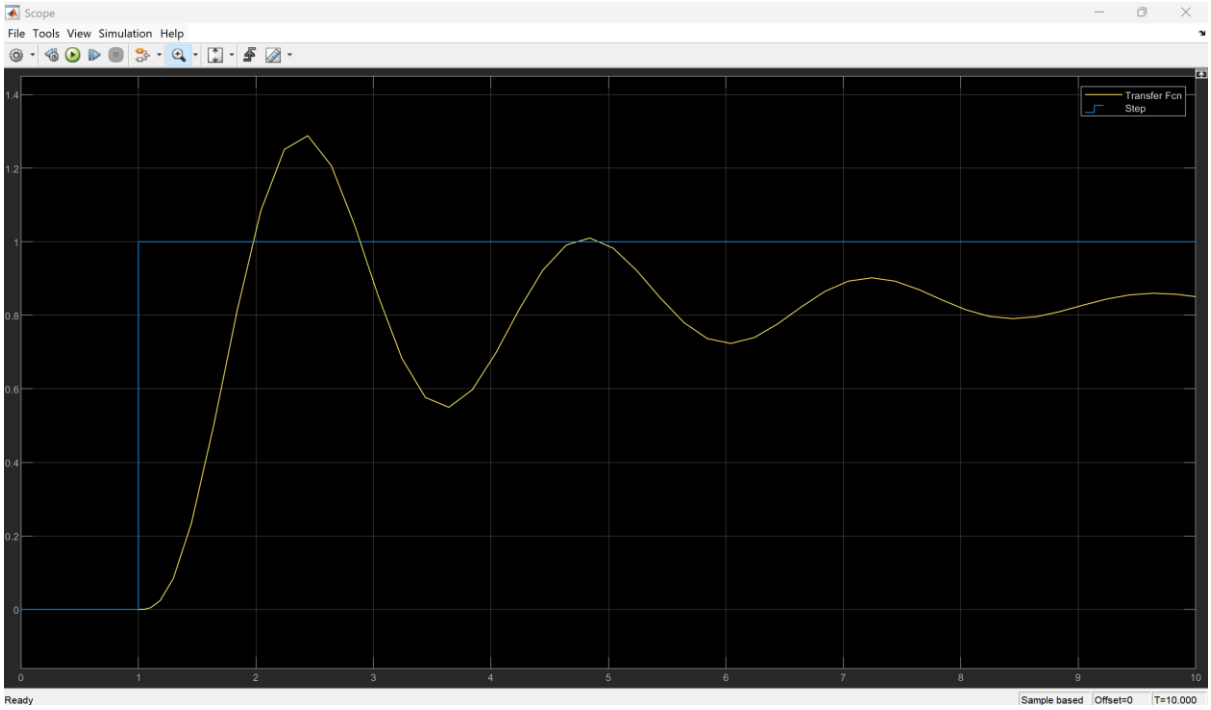






Using Ziegler Nichols, we can see the $K_{Pmax} = 10$ therefore $K_P = 0.5 * K_{Pmax} = 5$


PID1 * - Simulink academic use
SIMULATION
DEBUG
MODELING
FORMAT
APPS
SCOPE
New
Open
Save
Print
FILE
Library Browser
LIBRARY
Log Signals
Add Viewer
Signal Table
PREPARE
Stop Time
10
Normal
Fast Restart
Step Back
Run
Step Forward
Stop
SIMULATE
Data Inspector
Logic Analyzer
Bird's-Eye Scope
Simulation Manager
REVIEW RESULTS
Model Browser
Property Inspector
PID1
Quickly insert blocks by double clicking in the diagram and typing part of the block name. More information.
5
Proportional
0
1/s
Integral
0
Δu/Δt
Derivative
6/(s^3 + 6s^2 + 11s + 6)
Scope


Scope
File Tools View Simulation Help
Transfer Fcn
Step
1.4
1.2
1
0.8
0.6
0.4
0.2
0
0
1
2
3
4
5
6
7
8
9
10
Ready
Sample based
Offset=0
T=10.000

2. PI Controller

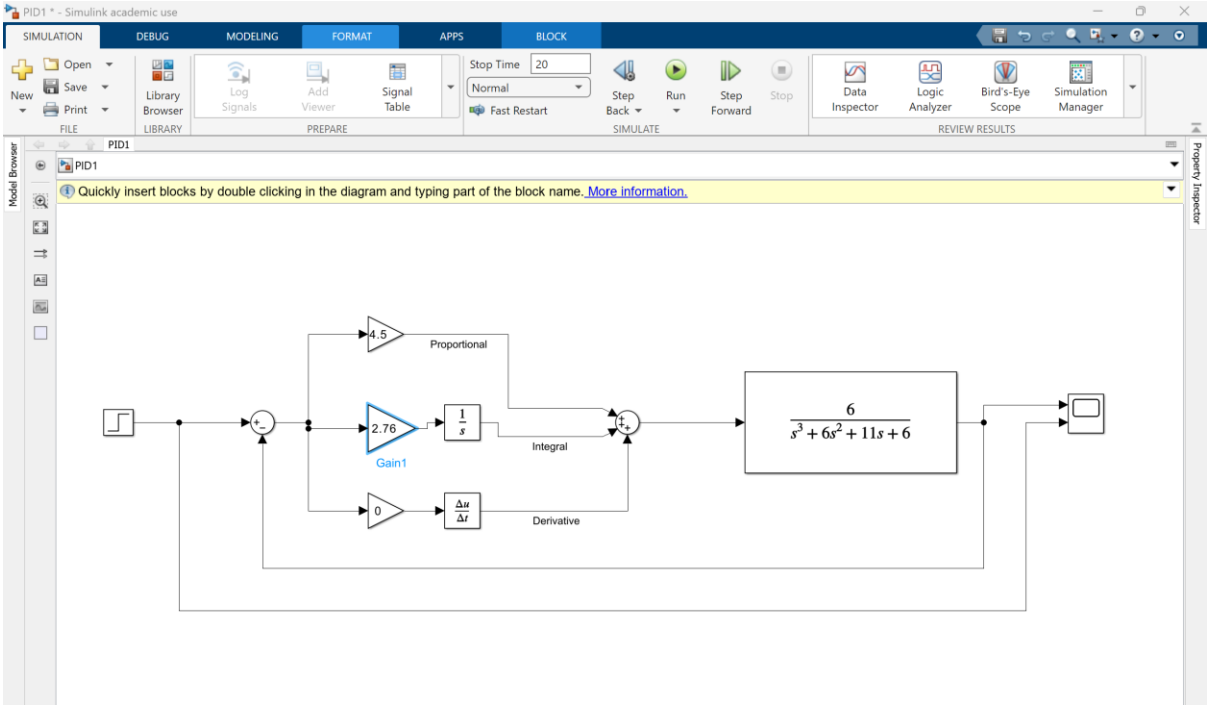
Using Ziegler-Nichols Method we Know the $K_{Pmax} = 10$ & Period of Oscillation $P_{Cr} = 1.95s$

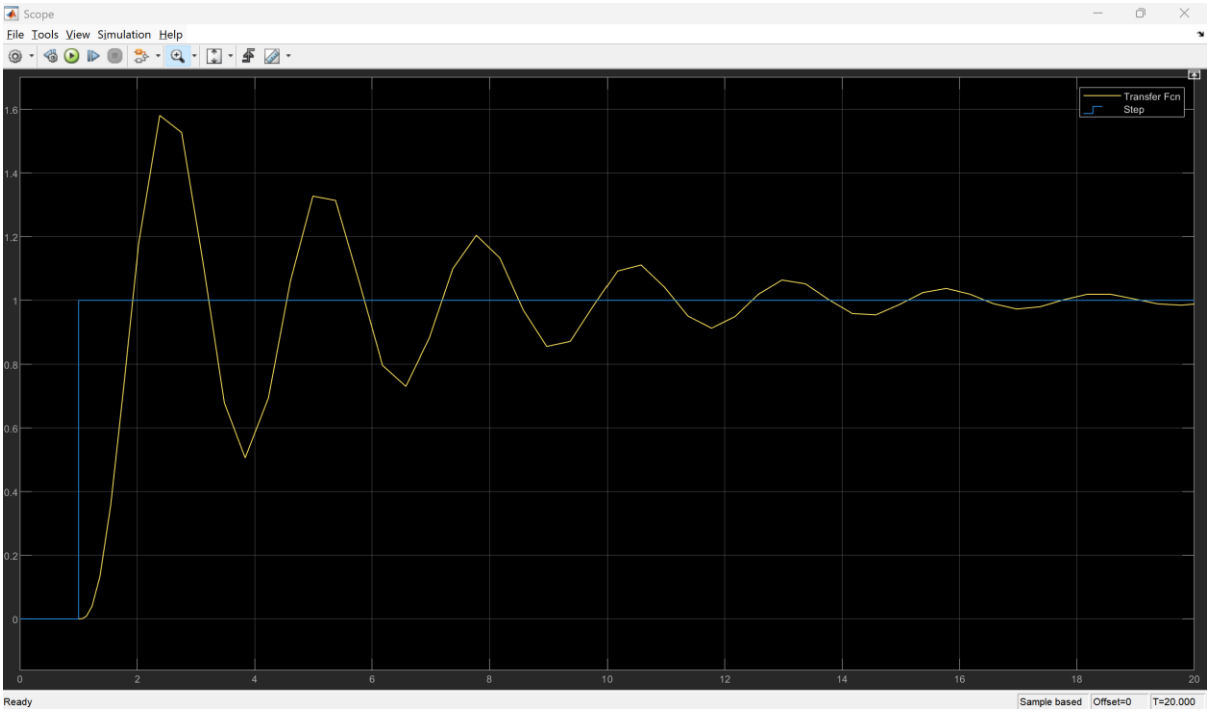
Therefore

$K_P = 0.45\ K_P = 4.5$

$T_I = P_{Cr} * 1/1.2 = 1.625$

$K_I = (0.54 * K_{Pmax})/P_{Cr} = 2.76$

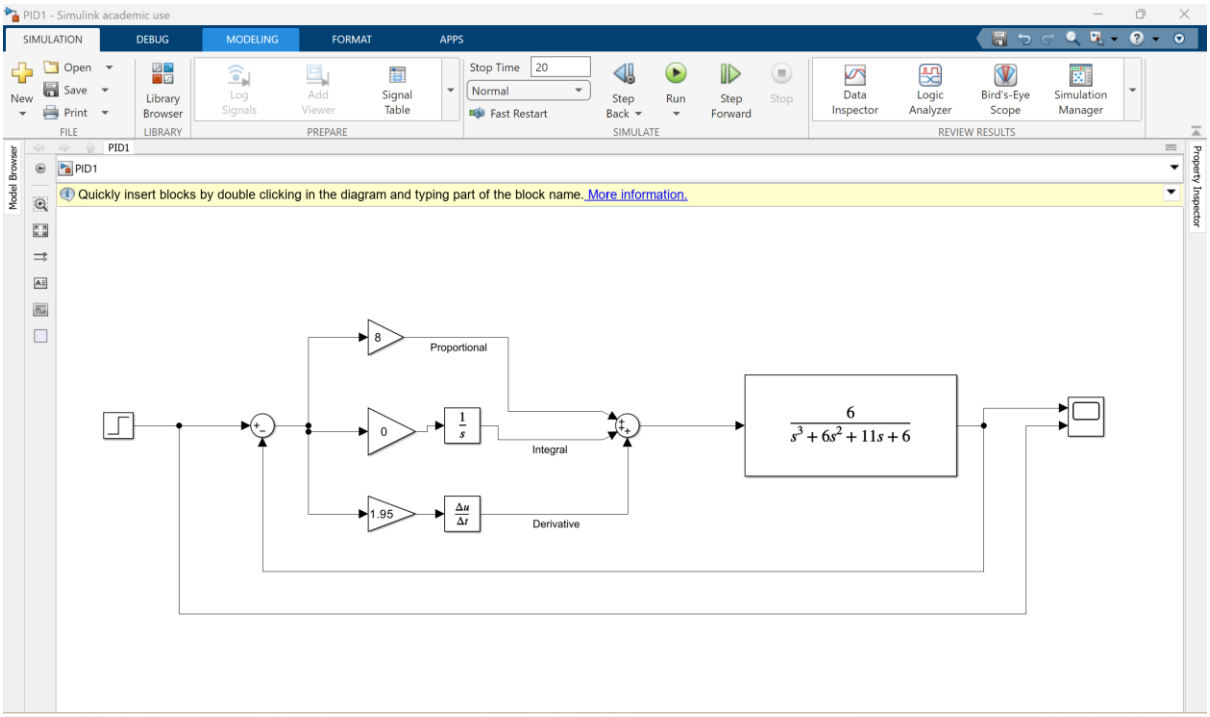


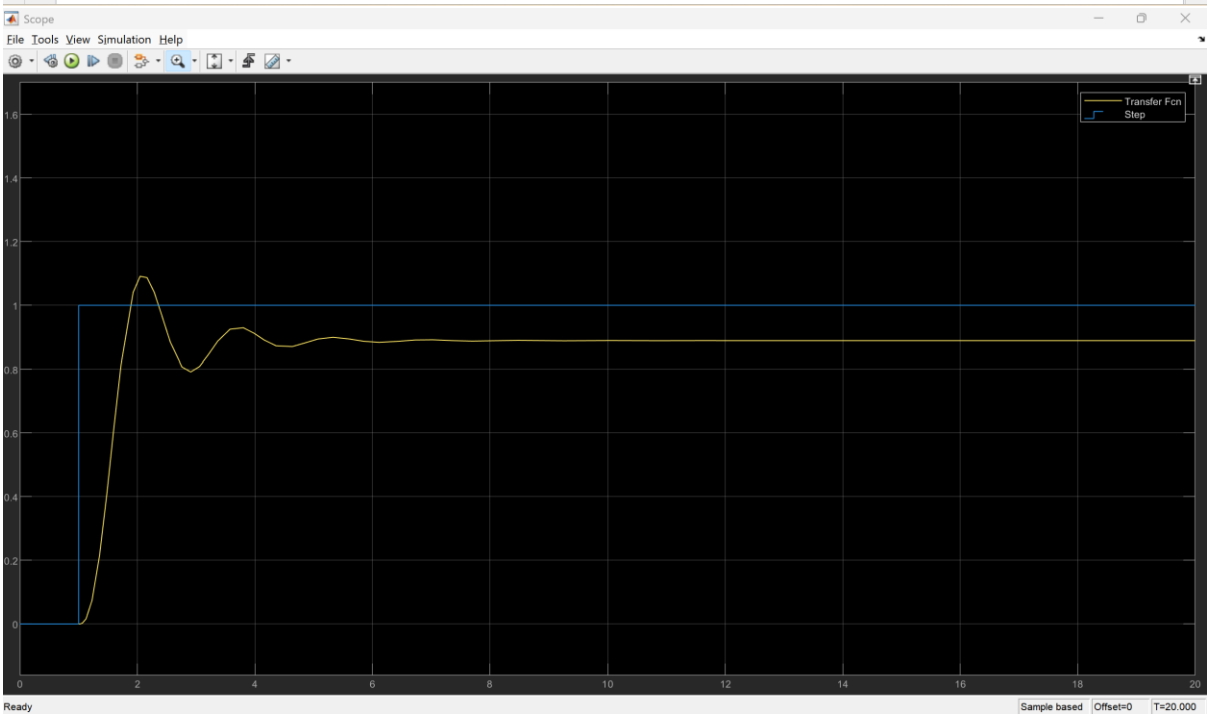


As we can see the Steady State Error has been reduced on the addition of the I term.

3. PD Controller

Using Ziegler-Nichols Method we Know the $K_{Pmax} = 10$ & Period of Oscillation $P_{Cr} = 1.95s$
Therefore
$K_P = 0.8\ K_P = 8$
$T_d = P_{Cr} * 0.125 = 0.24$
$K_I\ K_D = 0.10 * K_{Pmax} * P_{Cr} = 1.95$





As we can Observe the system's peak overshoot has reduced as well as oscillations

4. PID Controller

Using Ziegler-Nichols Method we Know the $K_{Pmax} = 10$ & Period of Oscillation $P_{Cr} = 1.95s$

Therefore

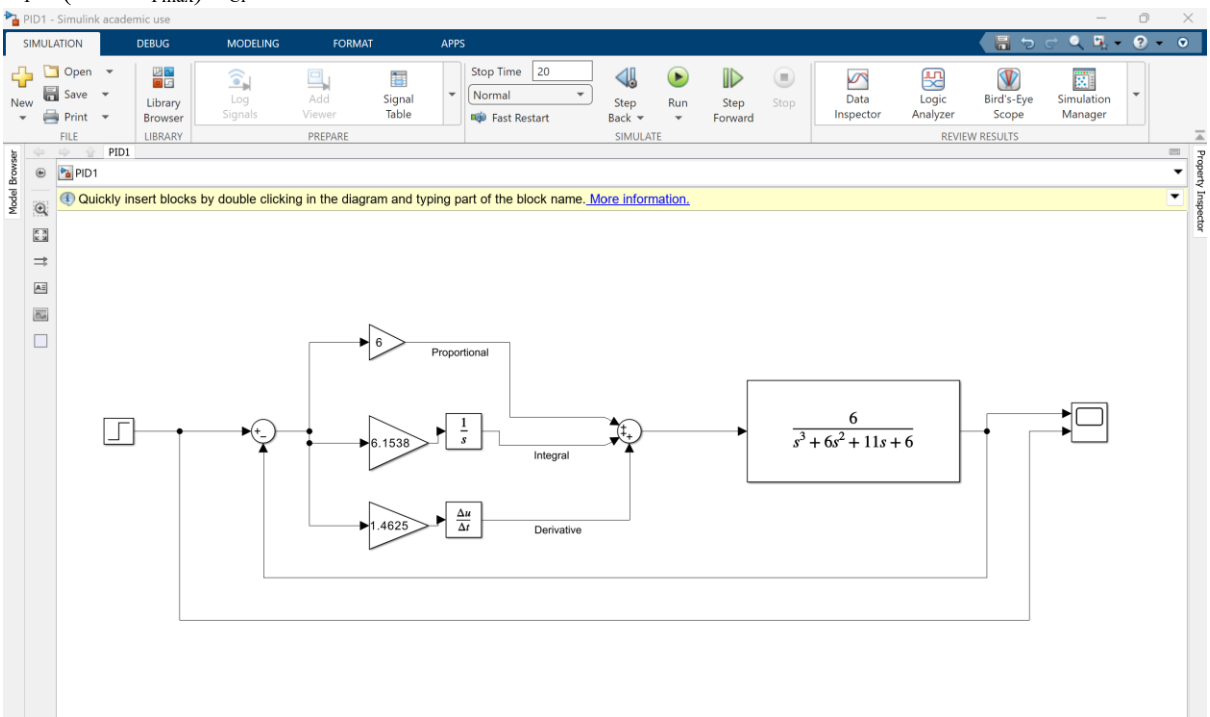
$K_P = 0.6\ K_P = 6$

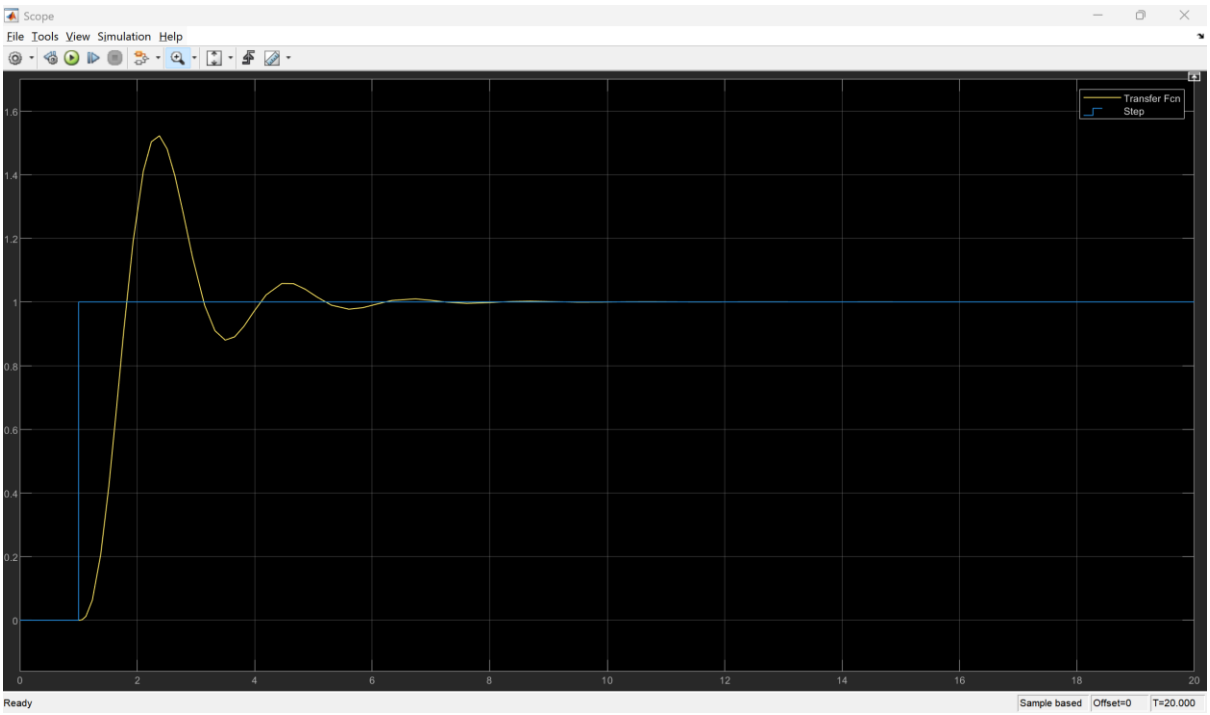
$T_I = P_{Cr} * 0.5 = 0.975$

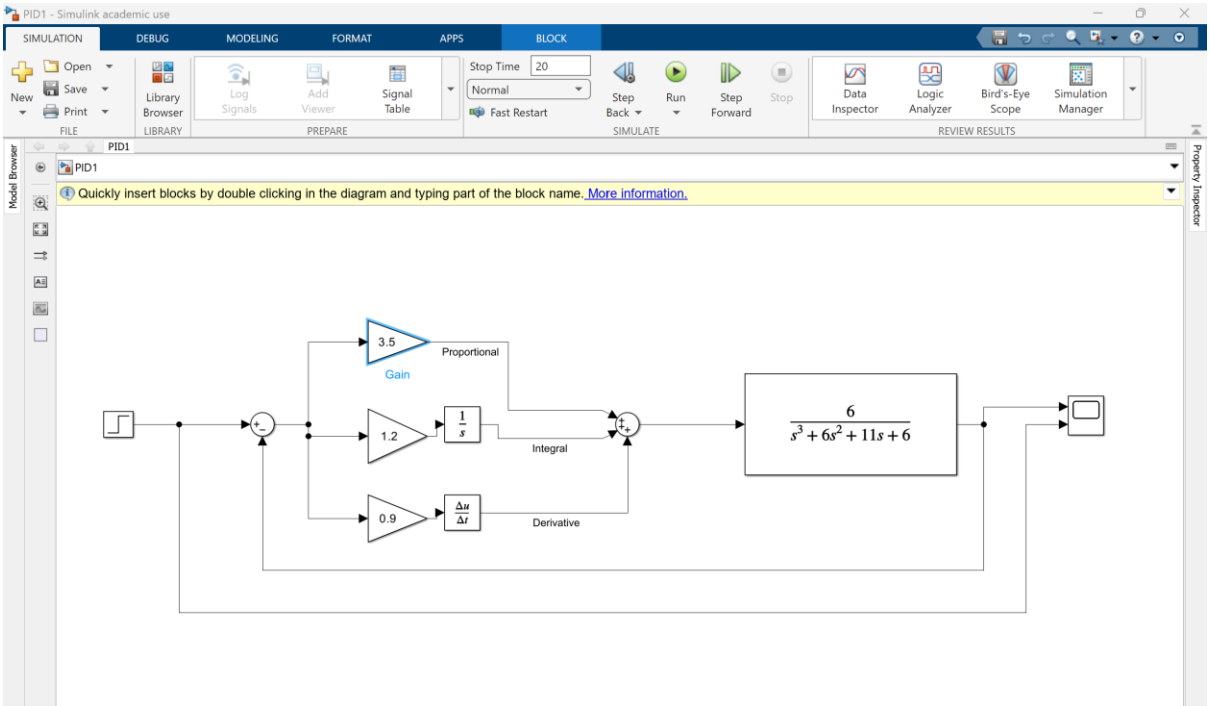
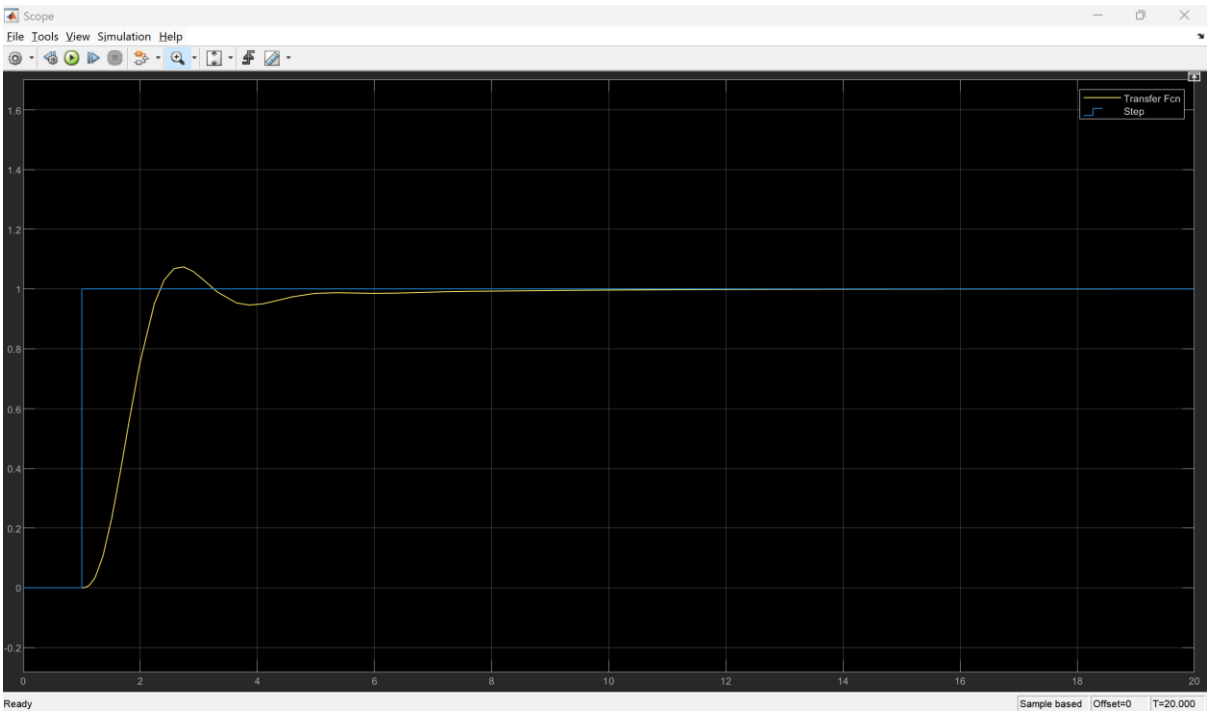
$T_d = P_{Cr} * 0.125 = 0.243$

$K_D = 0.075 * K_{Pmax} * P_{Cr} = 1.4625$

$K_I = (1.2 * K_{Pmax})/P_{Cr} = 6.1538$





As we can observe the system does not meet performance parameters on the basis of standard Ziegler Nichols tuning Parameters which gives an aggressive Overshoot and gain.

On manually tuning the parameters